

# Building a RAG demo takes an afternoon.

Building one that stays correct when a deploy fails halfway through is **a different engineering problem.**

|                                                                                      |                                                                              |                                                                         |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <div>17</div> <div>architecture decisions recorded, with rejected alternatives</div> | <div>87</div> <div>automated tests in the fast tier, run on every push</div> | <div>0</div> <div>metrics reported that nothing actually measured</div> |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|

- Multi-tenant

Durable ingestion

Cited answers

Cost attribution

Audit trail

512 MB, one worker

# A demo has one path. A system has all of them.

The tutorial version does upload → chunk → embed → answer.  
Every box works, and none of them says what happens when the process dies **between two of them**.

Changing one thing — upload returns **202 Accepted** in under a second, and the work continues without the request — forces almost everything else in this system to exist.

Durable queue. Atomic claim. Lease expiry. Bounded retry.  
Idempotent re-index. Tenant scoping on every read. An audit row in the same transaction.

**None of that is an AI problem.**

# The decision I'd defend hardest is one where I removed a feature.

## DIDN'T SHIP

### A confidence badge

~~High Confidence~~

It demos beautifully. Most RAG tutorials ship one.

But a label means picking cutoffs — above 0.7 is “High”. I had **no labelled dataset** to validate a single one of those numbers against.

## SHIPPED INSTEAD

### The measurements

|                 |         |
|-----------------|---------|
| Documents cited | 3       |
| Excerpts used   | 9 of 14 |
| Closest match   | 0.71    |
| Refusal floor   | 0.35    |

The score is **never shown without the floor it was compared against**. 0.71 means nothing alone.

An invented metric is worse than a missing one. A missing metric gets **questioned**. An invented one gets **quoted**.

# Postgres for the job queue. Not Redis.

Redis is faster and it is the conventional answer. I still chose the slower one.

## 01 A dropped job is data loss

Not a slow request. An uploaded document that silently never indexes is the worst failure this system has, because **nobody finds out**.

---

## 02 Postgres was already there

Already transactional, already backed up, already the source of truth. Redis would have been a **second stateful system** to operate for durability I had.

---

## 03 The claim is atomic by construction

FOR UPDATE SKIP LOCKED makes double-processing impossible, rather than unlikely. There is an integration test that proves it.

---

## 04 The queue is inspectable

It is a table. Debugging a stuck job at 3am is a SELECT, not a client library.

---

# Redis caches. That's all. A test enforces it.

A cache miss costs milliseconds. A lost queue job costs data. Those two failures should never share a system — and a note in a README does not stop the next person from mixing them.

So the boundary is **a test that fails the build if any file outside an explicit allow-list imports the cache.**

It asserts on **source code, not behaviour**, which is an unusual thing for a test to do.

I kept it because conventions rot and tests don't. I verified it actually discriminates by introducing a deliberate violation and watching it fail, then removing it.

## INCIDENT

# Six hours of silence.

The outage had nothing to do with the AI. Neither did my first fix.

### SYMPTOM

TLS completed. Request fully sent. **Zero bytes back** — across three attempts up to 90 seconds. Not a cold start.

### CAUSE

A migration created a Postgres enum type, then `create_table` created **the same type again**. It could never have succeeded. The first deploy left the type behind, so every boot afterwards hit the identical wall — and migrations run before the server starts.

```
DuplicateObject: type "ingestion_job_status" already exists
```

### MY FIRST FIX

**Also wrong.** I set `create_type=False` on a generic enum — a parameter that only exists on the dialect-specific type and is **silently ignored** there. Same error, one line lower.

### REAL FIX

The correct type, tolerant of the orphan already in production. This time I **compiled the DDL and counted the statements** before claiming it worked: two before, one after.

**Verify, don't reason — especially about your own fix.**

# What I measure — and what I refuse to.

The right column is the harder engineering decision.

MEASURED

Refusal rate

The one quality signal computable with no labels at all.

Uncited answers

The shape a hallucination takes here.

Cost per answer

Real provider tokens, attributed to org and document.

Retrieval traces

The query that actually ran, with scores.

REFUSED

Recall & precision

Need a labelled golden set. There isn't one, so the columns don't exist — and two tests assert they never appear.

Groundedness

Same reason.

“We benchmarked the reranker”

I evaluated one and declined — but on a handful of manual queries, not a golden set. Directionally right, not rigorous. Recorded that way.



# I won't call it production-ready.

It is **production-oriented**: built with production engineering principles, on free-tier infrastructure, with the unfinished parts named rather than smoothed over.

## BUILT

Durable ingestion, tenant isolation, RBAC, audit trail, cost tracking, correlation IDs, hybrid retrieval, grounded citations

## PARTIAL

**Worker shares the API process** — the free tier has no worker service, so ingestion load and request latency are coupled. Migration trigger is written down.

## PARTIAL

**Rate limits are in-process.** Correct at one worker; unenforceable at two.

## GAP

**No labelled golden set.** Seven tuning decisions are formally blocked on it — including the similarity floor that decides between “I don't know” and a confident fabrication.

A gap you have **named** is not the same as one you **haven't noticed**.



# What the demo never teaches

## 01 Building the AI feature was the easy part

Reliability, cost accountability and honest uncertainty took far longer — and none of them appear in a demo video.

---

## 02 A good decision names what it rejected

A list of technologies tells you nothing. A trade-off tells you how someone thinks.

---

## 03 The best thing I shipped was something I deleted

Removing the confidence badge made the product less impressive and more trustworthy.

---

## 04 A system that admits what it doesn't know is more useful than one that sounds certain

That is an engineering property, not a UX preference.

---

CLOSE

# Tell me where this breaks down.

Every trade-off in this system is written down, including the ones I reversed. I'd rather have the critique than the compliment.

## The two decisions I'm least sure about:

1. Postgres over Redis for the durable queue — what failure mode do you think I'm underweighting?
2. The similarity floor that triggers a refusal. It's currently a number with no validation behind it, and it decides whether a student gets an answer or a shrug.